

DFIR Orchestrator: System Architecture and Technical Specification (Mini-SPL)

- **Course:** SE 2215 - Design Patterns Lab
- **Project Name:** DFIR Orchestrator
- **Student Name:** Nafisa
- **Roll:** BSSE 1602
- **Student Name:** Miti Mugdho
- **Roll:** BSSE 1648
- **Technology Stack:** Java 21, JavaFX (FXML), SQLite (JDBC), Apache Maven, JUnit 5

1. Executive Summary and Problem Statement

Security Operations Center (SOC) teams and Digital Forensics and Incident Response (DFIR) specialists operate in high-pressure environments during active cyber breaches. Incident response requires rapid containment actions (such as network host isolation, malicious IP blocking, or credential invalidation) while simultaneously maintaining a forensically sound, legally defensible Chain of Custody for collected digital evidence.

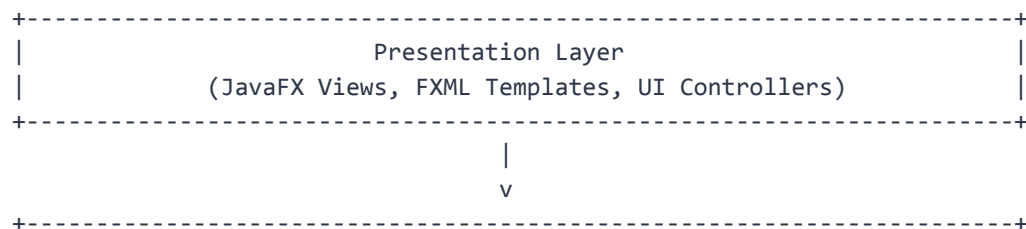
In practice, organizations face two critical operational failures:

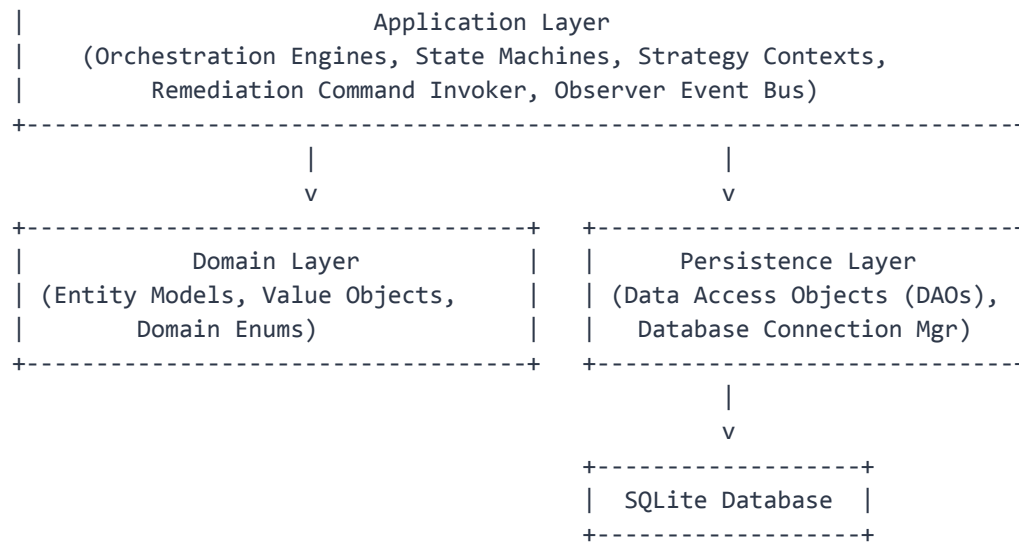
1. **Uncontained Threats:** Lack of standardized response lifecycles leads to missed phases (e.g., attempting eradication before containment is verified), resulting in reinfection or lateral movement.
2. **Evidence Inadmissibility:** Mishandling digital evidence (lack of cryptographic integrity verification, undocumented custody transfers, or unauthorized access to evidence under court hold) invalidates evidence for court proceedings.

DFIR Orchestrator is a desktop incident management and forensic orchestration system engineered to resolve both challenges. Rather than functioning as a passive set of database input forms, the system acts as an active, rule-enforcing orchestration engine. It enforces standardized response lifecycles, provides one-click reversible remediation actions backed by an immutable audit trail, calculates dynamic risk scores, guarantees evidence chain of custody integrity, and generates exportable forensic dossiers and analytics.

2. Multi-Tier Layered Architecture

The application is structured into four distinct layers, ensuring strict Separation of Concerns (SoC), modularity, and adherence to the Single Responsibility Principle.





2.1 Presentation Layer (`com.minispl.presentation`)

- **Components:** MainController, IncidentViewController, EvidenceViewController, AssetViewController, PlaybookViewController, ReportsViewController.
- **Responsibilities:** Renders JavaFX FXML interfaces, captures user input, translates UI actions into application service calls, and visualizes analytical charts and tables.
- **Decoupling:** Controllers implement the Refreshable interface and register with the application-layer event bus. Controllers do not execute raw database queries or direct business-rule validations; all mutations delegate to domain state machines and command invokers.

2.2 Application Layer (`com.minispl.application`)

- **Components:** State machines (IncidentStateMachine, EvidenceCustodyStateMachine), Playbook template runners (InvestigationEngine, RansomwarePlaybookEngine, PhishingPlaybookEngine), Remediation command infrastructure (CommandInvoker, RemediationCommandFactory), Strategy contexts (RiskScoringContext, HashingContext), and Event Publisher (IncidentEventPublisher).
- **Responsibilities:** Encapsulates the core business workflows, state transitions, algorithmic computations, and transactional rollback logic.

2.3 Domain Layer (`com.minispl.domain`)

- **Components:** Pure Plain Old Java Objects (Incident, EvidenceItem, Asset, User, Playbook, PlaybookStep, ActionAuditLog) and type-safe enumerations (IncidentStatus, IncidentSeverity, CustodyStatus, PlaybookPhase, AssetStatus, CriticalityTier, AuditStatus).
- **Responsibilities:** Models enterprise concepts and holds state. All model fields are encapsulated with private visibility and public accessor/mutator methods. The domain layer has zero dependencies on external frameworks or UI libraries.

2.4 Persistence Layer (`com.minispl.persistence`)

- **Components:** DatabaseManager, DatabaseSeeder, and isolated Data Access Objects (IncidentDAO, EvidenceDAO, AssetDAO, AuditLogDAO, PlaybookDAO, UserDAO).
- **Responsibilities:** Encapsulates all interactions with the persistent SQLite database (dfir_orchestrator.db). Enforces relational foreign-key integrity using PRAGMA foreign_keys = ON;;, executes prepared statements to prevent SQL injection, and maps relational rows into domain model objects. Business logic never contains inline SQL queries.

3. Design Patterns Implementation and Justifications

The system meaningfully integrates seven design patterns. Each pattern addresses a concrete architectural problem that cannot be effectively solved by procedural code.

3.1 Observer Pattern (Behavioral)

- **Problem Addressed:** When an incident changes state, when a remediation command executes, or when an asset is quarantined, multiple downstream components must react: the JavaFX UI tables must refresh, navigation badges must update, and an immutable entry must be recorded in SQLite. Hardcoding direct method calls from the state machines into UI controllers and persistence classes creates brittle, cyclic dependencies and violates the Single Responsibility Principle.
- **Implementation:**
- IncidentEventPublisher acts as a centralized Subject.
- IncidentEventListener defines the observer interface with the onIncidentEvent(IncidentEvent event) method.
- AuditTrailListener acts as a background subscriber that automatically persists state transition audit logs to SQLite.
- UI controllers (IncidentViewController, EvidenceViewController, AssetViewController) subscribe to receive updates and dispatch UI re-rendering on the JavaFX application thread using Platform.runLater.
- **Alternative Considered:** Direct invocations from within IncidentStateMachine. Rejected because it tightly binds domain logic to JavaFX presentation classes, preventing headless unit testing and modular reuse.

3.2 Template Method Pattern (Behavioral)

- **Problem Addressed:** Incident response standards (such as NIST SP 800-61) mandate an invariant five-phase investigation lifecycle: **Triage -> Containment -> Evidence Collection -> Eradication -> Post-Mortem**. Analysts cannot skip phases or alter the sequence. However, specific threats require fundamentally different concrete actions: a ransomware case requires shadow copy verification and file decryption, whereas a phishing case requires mail server purging and credential revocation.
- **Implementation:**

- InvestigationEngine is an abstract base class defining the invariant template method executePlaybook(Incident incident). This method controls the sequential execution loop, validates prerequisites, and invokes phase hooks (beforeInvestigation, onPhaseCompleted, afterInvestigation).
- RansomwarePlaybookEngine and PhishingPlaybookEngine extend InvestigationEngine and implement the abstract method executePhase(PlaybookPhase phase, Incident incident).
- **Alternative Considered:** Strategy Pattern. Rejected because Strategy allows algorithms to vary completely without preserving an invariant skeleton. Template Method enforces the required fixed phase sequence while delegating only the concrete phase steps to specialized subclasses.

3.3 State Pattern (Behavioral)

- **Problem Addressed:** Both incidents and forensic evidence items follow strict, legally governed lifecycles.
- Incident Lifecycle: NEW -> TRIAGED -> CONTAINED -> CLOSED.
- Evidence Custody Lifecycle: SEIZED -> IN_ANALYSIS -> COURT_HOLD -> ARCHIVED.

Transitions must adhere to strict rules (for example, evidence under COURT_HOLD cannot be directly archived without release, and an incident cannot be closed before containment). Procedural if-else or switch-case statements scattered across controllers lead to code duplication, high cyclomatic complexity, and invalid state corruptions.

- **Implementation:**
- Context classes: IncidentStateMachine and EvidenceCustodyStateMachine.
- State interfaces: IncidentState and CustodyState.
- Concrete state classes implement allowed transitions and reject invalid actions with descriptive domain exceptions (e.g., IllegalStateException). State changes are committed atomically to SQLite and broadcast via the event bus.
- **Alternative Considered:** Status flag enumerations checked inside controller event handlers. Rejected because adding new states or altering legal paths requires modifying every controller, violating the Open/Closed Principle.

3.4 Strategy Pattern (Behavioral)

- **Problem Addressed:** The system requires interchangeable algorithms for two distinct domain operations:
3. **Dynamic Risk Scoring:** Evaluating incident risk using either the NIST SP 800-61 criticality-threat matrix or the CVSS v3.1 mathematical severity scoring model.
 4. **Cryptographic Integrity Verification:** Generating cryptographic file digests using SHA-256 (standard speed) or SHA-512 (high-security court hold requirements).
- **Implementation:**
 - RiskScoringStrategy defines the risk calculation contract, implemented by NistRiskScoringStrategy and CvssRiskScoringStrategy. RiskScoringContext allows runtime strategy swapping.
 - HashingStrategy defines the file hashing contract, implemented by Sha256HashingStrategy and Sha512HashingStrategy. HashingContext computes hashes directly from disk files.

- **Alternative Considered:** Conditional flags (boolean useCvss, boolean useSha512). Rejected because adding future industry standards (e.g., FAIR quantitative risk modeling or BLAKE3 hashing) would require modifying existing classes rather than simply adding a new strategy implementation.

3.5 Command Pattern (Behavioral)

- **Problem Addressed:** SOC analysts execute active containment operations: isolating an endpoint from the network, blocking a malicious IP address at the firewall, or revoking active credentials. These actions must be encapsulated as first-class objects so they can be logged into an audit ledger with full parameter tracking, executed asynchronously, and undone via a single-click rollback (undo()).
- **Implementation:**
- RemediationCommand defines execute(), undo(), getCommandType(), and getParametersAsJson().
- Concrete commands (IsolateHostCommand, BlockIPCommand, RevokeCredentialsCommand) extend AbstractRemediationCommand.
- CommandInvoker executes commands, records execution status in action_audit_logs, and orchestrates rollback logic by restoring original asset and network states.
- **Alternative Considered:** Direct database update queries invoked from controller button listeners. Rejected because actions would lack parameter auditing, structured execution history, and automated reversal capabilities.

3.6 Factory Method Pattern (Creational)

- **Problem Addressed:** During playbook execution or audit log rollbacks, the application must instantiate the correct RemediationCommand or InvestigationEngine dynamically based on stored string identifiers (e.g., threat types or stored JSON parameters) without coupling the invoker to concrete implementation classes.
- **Implementation:**
- RemediationCommandFactory parses command types and JSON parameters to construct appropriate RemediationCommand instances.
- PlaybookEngineFactory instantiates the appropriate InvestigationEngine (RansomwarePlaybookEngine or PhishingPlaybookEngine) based on threat classification.
- **Alternative Considered:** Instantiating concrete classes via new operators inside UI controllers. Rejected because introducing a new command or playbook would require editing multiple controllers.

3.7 Singleton Pattern (Creational)

- **Problem Addressed:** Database connections and the application-wide event bus require centralized management to prevent resource leaks, connection contention, and conflicting event dispatchers.
- **Implementation:**
- DatabaseManager manages a single, persistent SQLite connection instance with enforced thread synchronization.

- IncidentEventPublisher maintains a single application-wide subscriber registry for decoupled event publishing.
- **Alternative Considered:** Passing connection and publisher instances manually through constructors across all classes. Rejected due to excessive constructor parameter bloat across presentation and application layers.

4. Core System Capabilities and Workflows

4.1 Workflow 1: Incident Intake, Dynamic Risk Scoring, and Triage

5. An analyst registers an incident, specifying title, threat type, initial severity, and affected endpoint asset.
6. The system queries the affected asset's criticality tier from AssetDAO.
7. The selected RiskScoringStrategy dynamically computes a normalized priority score (0.0 - 100.0) based on asset criticality, threat vector multipliers, and severity weights.
8. The analyst moves the incident into TRIAGED status via IncidentStateMachine.triage(), recording the analyst ID and broadcasting the state change.

4.2 Workflow 2: Guided 5-Phase Playbook Execution

9. The analyst initiates playbook execution for a triaged case.
10. PlaybookEngineFactory resolves the matching threat engine (RansomwarePlaybookEngine or PhishingPlaybookEngine).
11. The executePlaybook template method executes through the mandatory phases:
 - **Phase 1 (Triage):** Verifies threat indicators and file hashes.
 - **Phase 2 (Containment):** Executes host isolation and network blocking commands; advances incident status to CONTAINED.
 - **Phase 3 (Evidence Collection):** Acquires live memory dumps, forensic disk images, and network captures; logs evidence records into the database.
 - **Phase 4 (Eradication):** Identifies and removes malicious persistence mechanisms.
 - **Phase 5 (Post-Mortem):** Compiles phase results, records lessons learned, and transitions incident status to CLOSED.
12. An InvestigationReport object is compiled, detailing per-step success status and execution duration.

4.3 Workflow 3: Reversible Remediation Actions and Rollback

13. An analyst triggers a containment action (e.g., IsolateHostCommand for host DC-CORP-01).
14. CommandInvoker executes the command, changing the asset status from ONLINE to ISOLATED in the persistence layer.
15. The invoker serializes command parameters into JSON and records an audit log entry in action_audit_logs marked as can_undo = 1, status = 'EXECUTED'.
16. If the containment action is later deemed a false alarm, the analyst clicks 'Rollback Selected' in the Audit Console.

- 17. `CommandInvoker.undoByAuditLogId()` reads the record, re-creates the command via `RemediationCommandFactory`, executes `undo()`, restores the asset status to `ONLINE`, and updates the audit log status to `ROLLED_BACK`.

4.4 Workflow 4: Forensic Evidence Integrity and Chain of Custody

- 18. Physical or digital evidence items are cataloged in the Evidence Console and linked to an incident, source asset, and custodian.
- 19. An examiner uploads the disk file to the cryptographic verification tool.
- 20. `HashingContext` computes the cryptographic digest (SHA-256 or SHA-512) and compares it with the database baseline to certify zero tampering.
- 21. The item transitions through legal custody states (`SEIZED` -> `IN_ANALYSIS` -> `COURT_HOLD` -> `ARCHIVED`).
- 22. Custody transfers are guarded: evidence on `COURT_HOLD` cannot be released or archived without authorized custodian approval.

4.5 Workflow 5: Analytical Reporting, MTTC, and Export

- 23. `ReportsViewController` scans all contained and closed incidents.
- 24. The exact **Mean Time to Contain (MTTC)** is calculated in minutes by computing the time duration between incident creation and containment state transition audit logs.
- 25. Analysts can export a full Markdown **Forensic Dossier** summarizing incident details, playbook phases, custody history, and audit ledger.
- 26. Analysts can export an **Audit CSV Log** for external compliance reviews.

5. Database Schema and Persistence Architecture

The persistent SQLite database (`dfir_orchestrator.db`) is structured with seven normalized relational tables. Foreign key enforcement is guaranteed via connection-level pragmas.

5.1 Relational Schema Summary

Table Name	Primary Key	Foreign Keys	Key Columns	Purpose
users	id	None	full_name, role, email, created_at	Stores security personnel, analysts, and custodians with role-based access.
assets	id	None	hostname, ip_address, criticality_tier, status, description	Tracks infrastructure endpoints and their network quarantine state.

incidents	id	asset_id -> assets(id) assigned_analyst_id -> users(id) playbook_id -> playbooks(id)	title, threat_type, severity, status, risk_score, current_phase, created_at, closed_at	Core incident entity managing case status and risk scores.
evidence_items	id	incident_id -> incidents(id) source_asset_id -> assets(id) current_custodian_i d -> users(id)	evidence_name, evidence_type, file_hash, custody_status, collected_at	Manages chain of custody, cryptographic digests, and evidence status.
playbooks	id	None	threat_type, description, created_at	Defines available response procedure templates.
playbook_steps	id	playbook_id -> playbooks(id)	phase, sequence_order, step_name, action_type, pass_fail_criteria	Stores sequential phase steps and pass/fail criteria per playbook.
action_audit_logs	id	incident_id -> incidents(id) executed_by_id -> users(id)	command_type, parameters, timestamp, can_undo, status	Immutable audit ledger tracking all automated remediation commands and rollbacks.

5.2 Automated Seeding

On initial launch, DatabaseSeeder.seedIfEmpty() checks if tables contain existing records. If empty, it populates the database with realistic baseline operational data:

- 3 User profiles (Senior Analyst, Analyst, Admin)
- 4 Infrastructure assets (Domain Controller, Web Portal, Workstation, Database)
- 2 Complete Playbooks with 11 sequential steps (Ransomware and Phishing)
- Active sample incidents, cataloged evidence artifacts, and baseline audit entries.

6. Verification and Testing Architecture

The codebase includes an automated test suite comprising **52 unit and integration tests** built with JUnit 5. All tests execute independently without requiring an active graphical desktop environment.

6.1 Test Suite Breakdown

Test Suite Class	Tests	Focus Area	Verification Goals
ObserverPatternTest	5	Event Publishing & Subscriptions	Verifies listener notifications, decoupled audit persistence, and multi-observer independence.
TemplateMethodPlaybook Test	5	Playbook Execution Lifecycle	Confirms invariant 5-phase sequencing, threat-specific branching, and closure transitions.
IncidentStateMachineTest	6	Incident Lifecycle Guards	Tests legal transitions (NEW -> TRIAGED -> CONTAINED -> CLOSED) and verifies rejection of invalid operations.
EvidenceCustodyStateMachineTest	3	Chain of Custody Integrity	Verifies court hold enforcement and prevents unauthorized analysis or premature archiving.
RiskScoringStrategyTest	3	Strategy Pattern Scoring	Confirms calculation accuracy of NIST criticality matrices versus CVSS mathematical base scores.
HashingStrategyTest	3	Cryptographic Hashing	Verifies SHA-256 and SHA-512 digest accuracy across known file inputs.
RemediationCommandAndFactoryTest	5	Command Execution and Rollback	Tests host isolation, IP blocking, credential revocation, factory instantiation, and rollback restoration.
AdditionalWorkflowsTest	5	End-to-End Orchestration	Validates combined incident-evidence workflows, file verification, and markdown report generation.

MTTCCalculationTest	4	Analytics & Metrics	Verifies Mean Time to Contain mathematical calculations against synthetic audit log timestamps.
DatabaseAndDAOTest	7	Persistence Layer CRUD	Validates SQLite constraint enforcement, foreign key cascades, and DAO mapping accuracy.
FXMLLoadingTest	6	Presentation Layer Loading	Uses headless JavaFX platform initialization to ensure all six FXML view files load without hierarchy errors.

7. Future Extensibility Analysis

The architecture is designed to accommodate future organizational and technical requirements with minimal friction and zero regressions:

27. Adding New Incident Threat Types (e.g., Insider Threat, DDoS):

Subclass InvestigationEngine, implement executePhase, register the class in PlaybookEngineFactory, and insert step templates into playbook_steps. Zero modifications are required in existing Ransomware or Phishing engines.

28. Adding New Remediation Actions (e.g., SnapshotVMCommand, TerminateProcessCommand):

Implement the RemediationCommand interface, provide execute() and undo() logic, and add the command type mapping to RemediationCommandFactory. CommandInvoker and the audit rollback engine remain completely untouched.

29. Adding New Risk Scoring Models (e.g., FAIR Quantitative Model):

Implement RiskScoringStrategy and pass it to RiskScoringContext. Controllers and database tables require no code adjustments.

30. Adding New Cryptographic Standards (e.g., SHA-3, BLAKE3):

Implement HashingStrategy and configure it in HashingContext.

31. Tiered Incident Escalation (Tier 1 Triage -> Senior Analyst Review):

The Chain of Responsibility pattern can be cleanly layered into the Triage phase without modifying the underlying state machines.